// [SYSTEM_STATUS]: ACTIVE

Aegis-IoT

Neural Network Intelligence Platform

IoT Simulation · Federated Learning · Reinforcement Learning

Course: Computer Networks — Final Project Report

Students: Zain Ul Abdeen (2023773)

Hashir Awaiz (2023429)

Platform: AnyLogic 8.9.7 | Java 8+ | Python 3.x | React/Vite

Date: May 2026

AnyLogic • Scikit-Learn • m2cgen • Q-Learning • FedAvg • OneClassSVM • React/Vite • Java2D

v2.0.0 // PRODUCTION

Contents

Chapter 1

Introduction

1.1 Project Overview

The Internet of Things (IoT) connects large numbers of heterogeneous, resource-constrained devices and produces traffic that is bursty, noisy, and security sensitive. In this setting, simulation is valuable not only for capacity planning, but also for evaluating prediction and control strategies before deployment.

Aegis-IoT is an AnyLogic-based IoT network simulation augmented with embedded intelligence for (i) latency prediction, (ii) anomaly detection, (iii) short-horizon forecasting, and (iv) adaptive load management. The platform exports telemetry and supports real-time observability through a desktop Network Operations Center (NOC) dashboard and a lightweight web dashboard.

1.2 Problem Statement

Many IoT simulations remain primarily descriptive: they replay traffic but do not close the loop with inline analytics and control. This limits the ability to:

- anticipate congestion (predictive latency estimation),
- flag abnormal traffic patterns quickly (inline anomaly scoring), and
- evaluate privacy-preserving learning setups (distributed training without raw data centralization).

1.3 Objectives

The project objectives are to:

1. model MQTT-style packet flows using an agent-based discrete-event simulation,
2. train and deploy models for latency prediction, anomaly detection, and short-horizon forecasting,
3. integrate a tabular Q-learning policy for adaptive congestion control,
4. prototype a FedAvg-style federated learning loop across simulated edge nodes, and

5. provide real-time visualization and offline analysis via desktop and web dashboards.

1.4 Scope and Contributions

Table 1.1 summarizes the main deliverables of the project.

Table 1.1: Summary of Technical Contributions

Contribution	**Description**
Simulation core	AnyLogic model of an IoT sensor–gateway–cloud path with MQTT-like packet flows and structured telemetry.
Embedded prediction	Supervised models for latency estimation and short-horizon forecasting integrated into the simulation loop.
Anomaly detection	Unsupervised novelty scoring for highlighting abnormal traffic consistent with DDoS-style behavior.
Adaptive control	Tabular Q-learning policy that adjusts processing behavior based on congestion state.
Privacy-aware training	Federated learning prototype (FedAvg) demonstrating distributed training without centralizing raw data.
Observability	Desktop NOC dashboards and a web analytics dashboard for time-series, distribution, and status views.
Reproducible tooling	Automated project mutation/injection workflow for repeatable integration of intelligence and UI features.

The deliverables include a reproducible injection workflow, exported Java predictors for in-simulation inference, and a multi-surface monitoring stack (desktop NOC + web analytics) designed to support both real-time operations and offline analysis. The project emphasizes practical integration: artifacts such as injector scripts, exported model code, and dashboard assets are provided to lower the barrier for replication and extension. These materials enable other researchers to reproduce the experiments, validate model behavior under controlled perturbations, and iterate on the learning and control components.

Chapter 2

Literature Review

This project combines established ideas from simulation, network analytics, and distributed learning into a single experimental platform.

2.1 IoT Simulation and MQTT

AnyLogic [3] supports agent-based and discrete-event modeling, making it suitable for simulating end-to-end IoT workflows with embedded logic. MQTT [2] is a widely used lightweight publish–subscribe protocol in IoT; modeling MQTT-style flows enables repeatable experiments around latency, burstiness, and anomalous traffic.

2.2 Learning for Prediction and Detection

For latency prediction, Random Forests [8] provide a strong baseline for non-linear regression on tabular telemetry features and are robust to noise. For anomaly detection without requiring labeled attacks, One-Class SVMs [7] learn a boundary around normal traffic and flag out-of-distribution observations.

To stress-test detection logic, DDoS behavior can be approximated by generating traffic that is statistically distant from normal operation; GANs [9] motivate this style of synthetic generation, while survey work [14] contextualizes common volumetric flooding patterns.

2.3 Adaptive Control and Distributed Training

Reinforcement learning provides a framework for adaptive control under uncertainty. Q-learning [6] is a simple, well-studied approach for learning a policy over discrete state–action spaces and is appropriate for prototyping congestion-aware decisions.

Federated Learning [5] enables collaborative model training without centralizing raw data. The FedAvg algorithm aggregates model updates from distributed clients, supporting experiments in privacy-preserving training and bandwidth reduction.

Digital Twin concepts [13] motivate mapping live telemetry into higher-level health indicators that support operator situational awareness.

2.4 Deployment in JVM-Based Simulations

Bridging Python-based model development and a JVM-based simulator motivates model-to-code approaches. The `m2cgen` framework [4] exports classical ML models into native source code, enabling inference inside the simulation runtime without a separate service boundary.

Table 2.1: Techniques Used in Aegis-IoT and Motivation

Technique	**Motivation in This Project**
AnyLogic simulation [3]	Rapid, agent-based experimentation with end-to-end IoT flows and inline logic.
MQTT modeling [2]	Realistic IoT traffic abstraction for latency and anomaly experiments.
Random Forest regression [8]	Non-linear latency prediction on telemetry features with good stability.
One-Class SVM [7]	Unsupervised novelty detection for attack-like or abnormal traffic.
Q-learning [6]	Simple adaptive policy learning for congestion-aware control.
FedAvg [5]	Privacy-aware distributed training prototype across edge nodes.
m2cgen [4]	In-process inference in the JVM without external serving infrastructure.

Chapter 3

System Architecture

3.1 High-Level Architecture

The Aegis-IoT platform is organized into five decoupled layers that connect simulation, intelligence, observability, distributed learning, and persistence. Figure 3.1 presents the complete system architecture and data-flow paths.

3.2 Simulation Model

The simulation is implemented in AnyLogic [3] using an agent-based discrete-event approach. Table 3.1 lists the principal agent types. Note that unlike a traditional multi-tier network, the AnyLogic model concentrates intelligence within the `Main` agent; gateway and cloud routing logic is embedded in the packet-processing hooks injected by the Injection Engine rather than modeled as separate agent classes.

Table 3.1: AnyLogic Agent Types

Agent	Role	Count
SensorDevice	Generates telemetry packets using dataset-derived distributions	Multiple
MQTTPacket	Carries network metrics (`packetSize`, `interArrival`, `flowDuration`) through the simulated path	Dynamic
Main	Orchestrates execution: hosts all four transpiled models, the RL controller, GAN-style stress injection, dashboard windows, and telemetry export	1

Each packet carries a small set of features used for analytics and visualization (Table 3.2).

Table 3.2: Packet-Level Network Metrics

Field	Type	Meaning
packet_size	double	Payload size (bytes)
inter_arrival	double	Time between arrivals (µs)
flow_duration	double	End-to-end flow duration (ms)

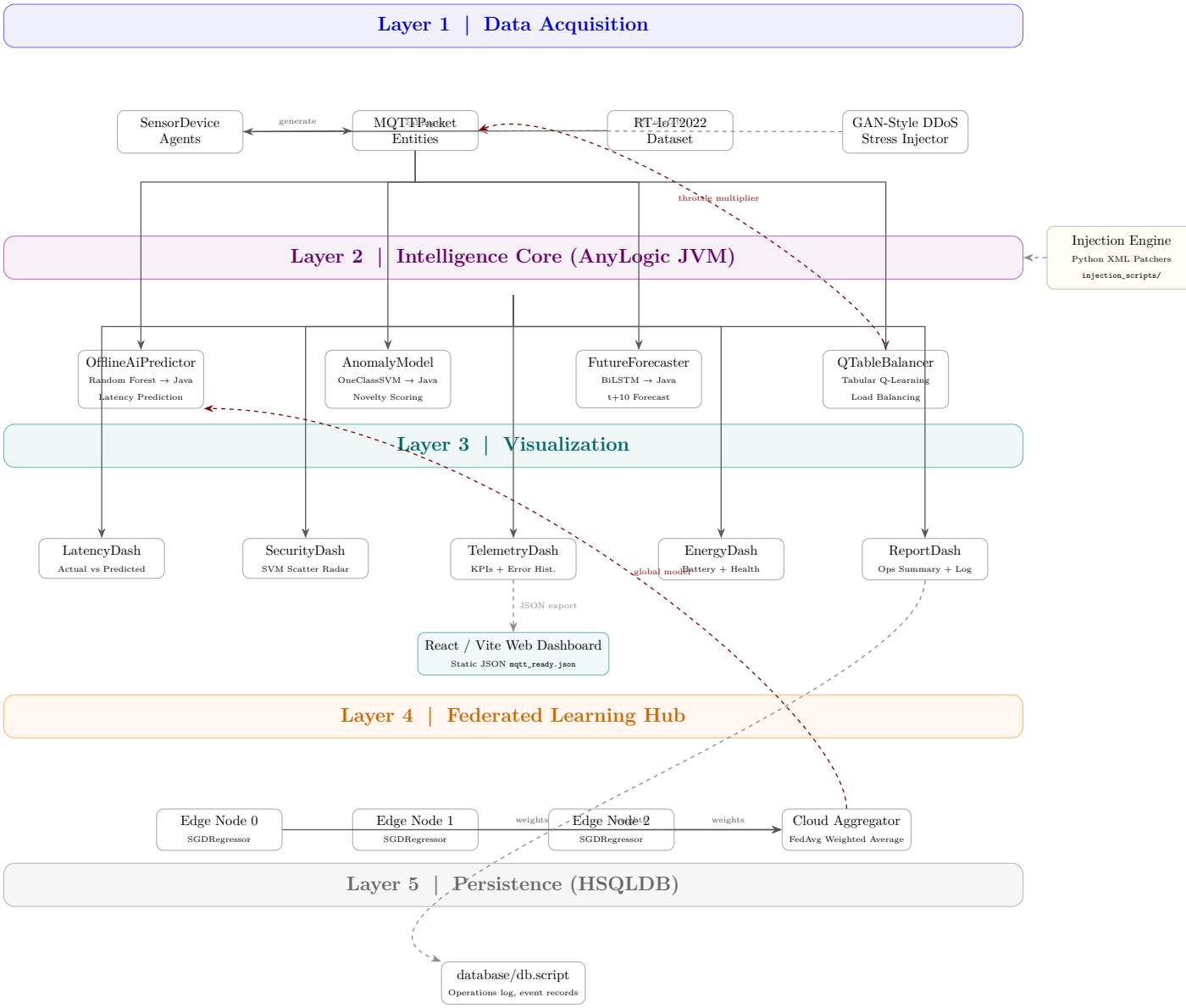

Figure 3.1: Complete Aegis-IoT system architecture: five layers from data acquisition through intelligence, visualization, federated learning, and persistence. Dashed arrows indicate asynchronous or offline data paths; red dashed arrows indicate feedback loops.

3.3 Processing Pipeline

When packets reach the processing sink inside `Main`, the simulation executes the following sequence:

1. **GAN-style stress injection:** with 5% probability, the packet's features are mutated to emulate volumetric DDoS traffic (large packet size, micro-burst inter-arrival, inflated flow duration). This synthetic adversarial perturbation validates the anomaly detector under attack-like conditions.
2. **RL congestion control:** the `QTableBalancer` maps the current queue proxy to a discrete state, selects an action (accelerate 0.5×, maintain 1.0×, or throttle 2.0×) via ϵ-greedy policy, applies the multiplier to `flow_duration`, and updates the Q-table using the temporal-difference rule.
3. **Model inference:**
   - `OfflineAiPredictor.score()` estimates latency from `(packetSize, interArrival)`.
   - `AnomalyModel.score()` computes a novelty score from `(packetSize, interArrival, flowDuration)`.
   - `FutureForecaster.score()` predicts latency 10 steps ahead using a BiLSTM-based sliding window over recent flow durations.
4. **Telemetry broadcast:** updated metrics are pushed to the five NOC dashboard windows and, on export, written as a JSON artifact consumed by the web dashboard.

3.4 Transpiled Model Inventory

Table 3.3 lists all models that are exported to pure Java via `m2cgen` and executed inline during simulation.

Table 3.3: Transpiled Java Models (Zero-Dependency Inference)

Java Class	Model & Purpose	Training Script
OfflineAiPredictor	Random Forest regressor for latency prediction using (packetSize, interArrival)	`train_latency_random_forest.py`
AnomalyModel	OneClassSVM (RBF kernel) for novelty scoring of DDoS-like traffic	`test_m2cgen4.py`
FutureForecaster	BiLSTM for autoregressive t+10 latency forecasting via sliding window	`train_forecaster.py`

3.5 Dataset

Experiments use RT-IoT2022 MQTT traffic [1]. After preprocessing, a cleaned dataset of 4,132 flow records is used to calibrate distributions and train models. Table 3.4 reports summary statistics for the core fields.

Table 3.4: Dataset Statistics (clean_mqtt_traffic.csv)

Feature	**Mean**	**Std Dev**	**Min**	**Max**
packetSize	7.71	2.14	3.43	15.00
interArrival	2,462,430	845,212	1,024	4,921,000
flowDuration	32.01	10.98	0.13	64.00

Chapter 4

Machine Learning Pipeline

4.1 Overview

The Aegis-IoT learning pipeline is designed for low-friction experimentation and deployment inside a JVM-based simulator. It follows three phases:

1. **Offline training:** models are trained from cleaned telemetry using standard ML tooling (scikit-learn) [10].
2. **Model export:** classical models are exported to native Java using model-to-code generation [4].
3. **In-simulation inference:** exported predictors execute inline during packet processing.

Figure 4.1 illustrates this three-phase workflow.

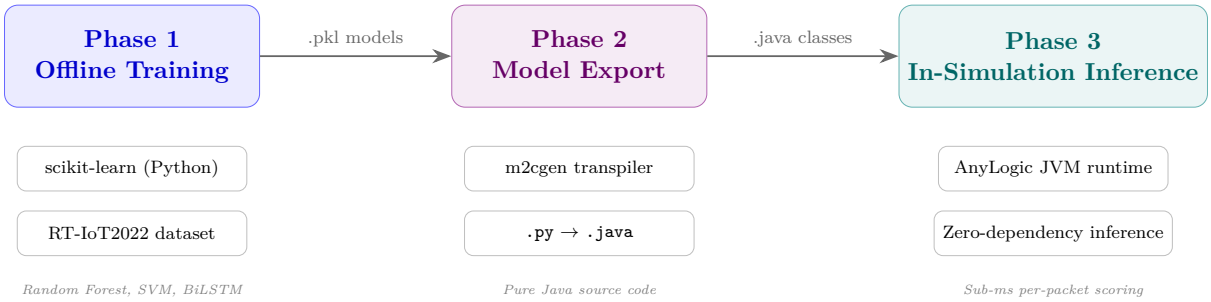

Figure 4.1: Three-phase ML pipeline: offline training in Python, model export to Java via m2cgen, and zero-dependency inference inside the AnyLogic JVM.

4.2 Data and Features

Experiments are based on the RT-IoT2022 dataset [1], cleaned into 4,132 packet-flow records. Features are derived from packet size, inter-arrival timing, and observed flow duration. Three feature views are used:

- **Latency prediction:** packetSize and interArrival.
- **Anomaly detection:** packetSize, interArrival, and flowDuration.

- **Forecasting:** sliding windows over recent flowDuration values.

4.3 Models Deployed

Table 4.1 summarizes the models used in the platform and how they are executed.

Table 4.1: Model Summary and Runtime Integration

Capability	Model and Inputs	Output
Latency prediction	Random Forest regressor (`OfflineAiPredictor`) using (packetSize, interArrival) [8]	Estimated flow duration
Anomaly scoring	One-Class SVM (`AnomalyModel`) using (packetSize, interArrival, flowDuration) [7]	Novelty score / flag
Short-horizon forecasting	BiLSTM forecaster over recent flowDuration windows, predicting t+10	Predicted future latency

4.4 Evaluation Metric

For regression tasks, performance is reported using Mean Absolute Error (MAE):

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{4.1}$$

4.5 Deployment Rationale

Exporting models into Java source enables in-process inference inside AnyLogic without external services, serialization steps, or cross-language RPC overhead. This keeps experiments reproducible and reduces engineering complexity for the simulation runtime.

This in-process approach minimizes inference latency for decision-making during packet processing, simplifies deployment by removing external serving infrastructure, and accelerates iteration for model updates. Tight coupling between simulator state and predictor inputs improves fidelity of offline-to-online evaluation, allowing more realistic validation of control strategies. Finally, shipping lightweight exported models reduces the operational footprint for demonstration and teaching scenarios.

Chapter 5

Reinforcement & Federated Learning

5.1 Tabular Q-Learning Load Balancer

5.1.1 Motivation

IoT traffic can change rapidly due to bursts and attack-like behavior. A static buffering or throttling policy is therefore difficult to tune across operating regimes. Reinforcement learning provides an adaptive control mechanism that learns which action is preferable for a given congestion state.

5.1.2 State–Action Design

The controller uses a discrete state space based on queue depth and a small action space of delay multipliers (Table 5.1).

Table 5.1: Q-Learning State–Action Space

Component	Values	Description
State Space		
Low	Queue < 10	Minimal congestion
Medium	$10 \leq$ Queue < 50	Moderate load
High	$50 \leq$ Queue < 100	Heavy congestion
Critical	Queue ≥ 100	Imminent overflow
Action Space		
Accelerate	$0.5\times$	Speed up processing
Maintain	$1.0\times$	No change
Throttle	$2.0\times$	Slow down processing

5.1.3 Reward and Update Rule

The immediate reward is defined to penalize large flow durations:

$$r(s, a) = -\text{flowDuration}(s, a) \tag{5.1}$$

Q-values are updated using the standard temporal-difference rule [6]:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right] \tag{5.2}$$

5.1.4 Integration

During packet processing, the simulation estimates a congestion state, selects an action using an ϵ-greedy policy, applies the multiplier, and updates the Q-table. Figure 5.1 illustrates this closed feedback loop.

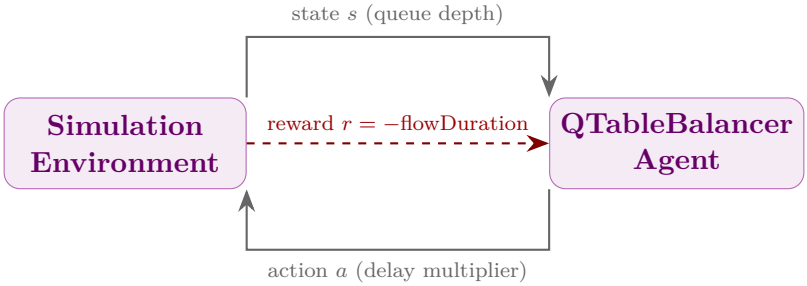

Figure 5.1: Closed-loop reinforcement learning: the simulation provides state and reward signals to the QTableBalancer, which selects a delay multiplier action applied to packet processing.

5.2 Federated Learning Pipeline

5.2.1 Motivation

Centralizing raw telemetry can be undesirable due to privacy and bandwidth constraints. Federated Learning mitigates this by training locally at the edge and sharing only model updates.

5.2.2 FedAvg Summary

The platform prototypes Federated Averaging (FedAvg) [5] in a simplified setting:

1. the cloud initializes and broadcasts a global model,
2. each edge node trains locally on its private data partition,
3. nodes upload model updates (not raw records), and
4. the cloud aggregates updates to form the next global model.

Figure 5.2 visualizes a single communication round.

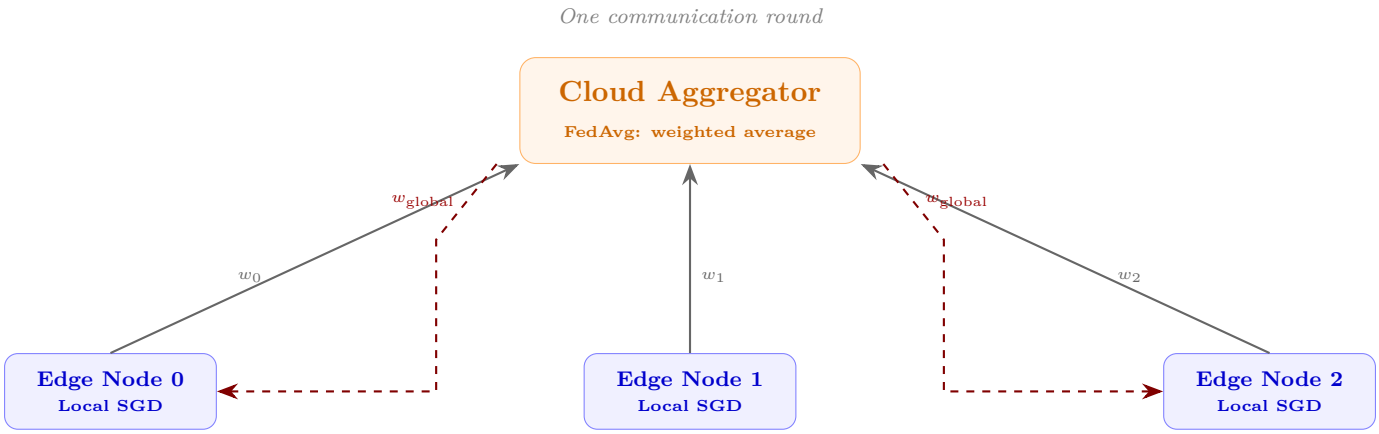

Figure 5.2: FedAvg communication round: edge nodes train locally on private data partitions, upload model weights to the cloud aggregator, and receive the updated global model.

The aggregation step can be expressed as:

$$w_{t+1} = \sum_{k=1}^{K} \frac{n_k}{n} w_{t+1}^k \tag{5.3}$$

where K is the number of clients, n_k the number of samples at client k, and n the total number of samples.

5.2.3 Prototype Configuration

The current prototype simulates $K = 3$ edge nodes, each holding a disjoint partition of the cleaned MQTT dataset. Local models are `SGDRegressor` instances trained for 5 local epochs per round. Aggregation runs for 10 communication rounds. Although this is a simplified setting, it demonstrates that the global model's error decreases monotonically across rounds, validating the FedAvg workflow without requiring raw data to leave the simulated edge.

Chapter 6

NOC Dashboard

This chapter presents the Network Operations Center (NOC) windows used for real-time monitoring and analysis. Each dashboard screenshot below is paired with a concise description that explains its purpose and key visual elements.

6.1 Design Philosophy

The simulator exposes live telemetry through a decoupled, multi-window Network Operations Center (NOC). Instead of a single monolithic UI, each dashboard focuses on a specific operator task (latency, security, system health, and energy/twin indicators). This reduces information overload and limits rendering overhead on the simulation engine.

6.2 Dashboard Overview

Table 6.1: NOC Dashboard Windows

Window	**Class**	**Primary Content**
Latency Hub	LatencyDash	Actual vs. predicted latency trends; short-horizon forecast curve
Security Radar	SecurityDash	Packet feature scatter with anomaly highlighting for rapid triage
Telemetry Matrix	TelemetryDash	KPI counters and error distribution summaries
Energy & Twin	EnergyDash	Battery depletion trend and composite health indicator
Report	ReportDash	Aggregated AI model summaries and consolidated experiment reports

6.3 Thread Safety and Performance

Dashboards are updated asynchronously to avoid blocking the discrete-event engine. UI updates are marshaled onto the Swing Event Dispatch Thread, while data collection remains in the simulation loop. To keep performance stable over long runs, charts operate on sliding windows (fixed recent history) rather than unbounded series.

6.4 Dashboard Screenshots

The following five windows illustrate the primary operator views. Each subsection below provides a short description followed by the corresponding screenshot and a concise caption.

6.4.1 Dashboard 1: Latency & Forecasting Hub

Description: This dashboard visualizes the temporal performance and predictive capabilities of the network simulation.

Live Latency Tracking: The top chart compares the actual network latency against the AI-predicted latency in real-time. It utilizes a Random Forest model (transpiled via m2cgen) to estimate current latency, showing a tight mapping during normal operations and occasional divergence during high-volatility spikes.

Time-Series Forecasting: The bottom chart utilizes a Bidirectional LSTM (BiLSTM) forecaster to predict future latency trends (t+10 steps ahead). This provides a forward-looking view to proactively anticipate network congestion or degradation before it impacts operations.

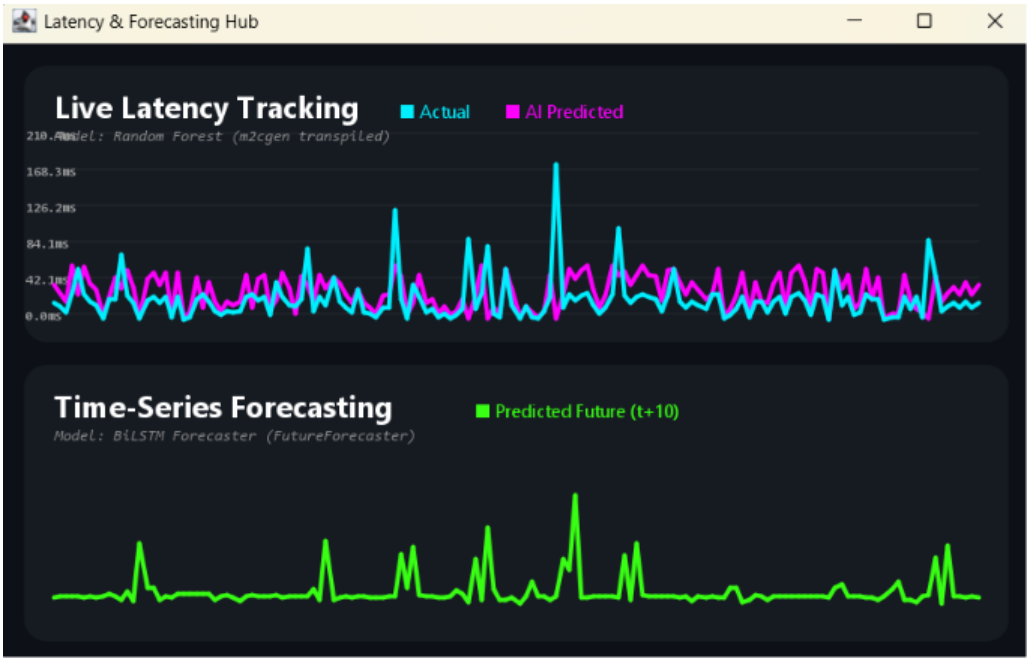

Figure 6.1: Real-Time Latency Tracking and Time-Series Forecasting using Random Forest and BiLSTM Architectures.

6.4.2 Dashboard 2: Security & Anomaly Radar

Description: This module acts as the primary intrusion detection mechanism, focusing on identifying malicious or abnormal network flows.

Defense Radar: The scatter plot maps network packets based on two key features: Packet Size (bytes) on the x-axis and Flow Duration (ms) on the y-axis.

Classification: Powered by a One-Class Support Vector Machine (SVM), the system clusters normal operational traffic (marked in blue) typically characterized by smaller packet sizes and lower durations. Outliers—representing potential threats like DDoS packets or data exfiltration attempts—are isolated and flagged as anomalies (marked with large red indicators).

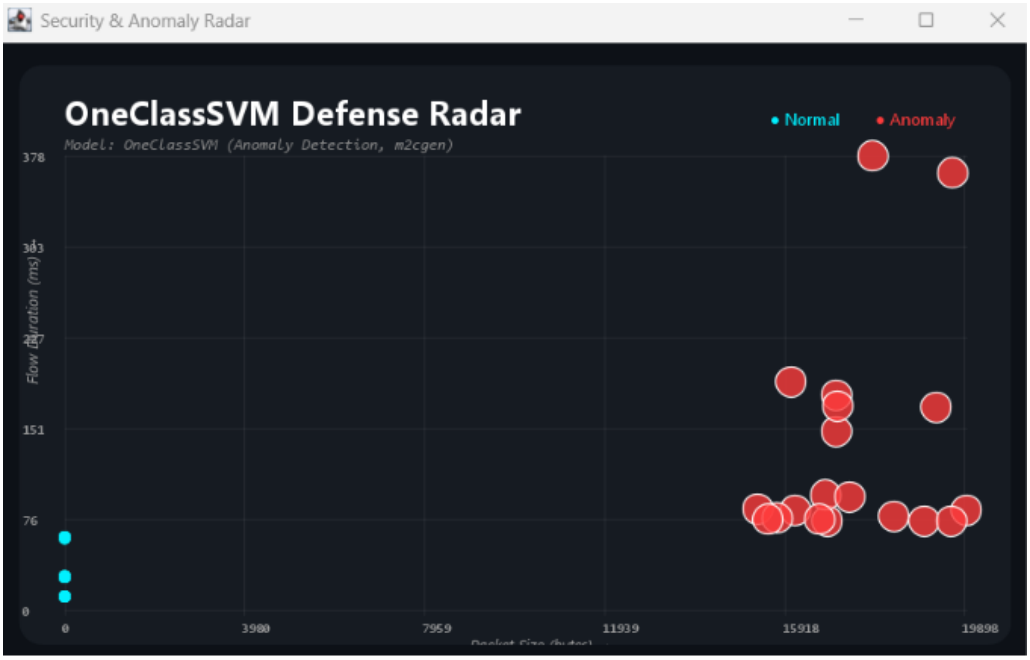

Figure 6.2: Unsupervised Anomaly Detection of Network Traffic via One-Class SVM.

6.4.3 Dashboard 3: Telemetry Matrix

Description: This interface provides a deep dive into the inner workings of the active AI models and the simulation's adversarial environment.

Advanced AI Core: The left panel tracks the Reinforcement Learning (Q-Learning) agent's reward stability and policy optimization, indicating how well the agent is adapting to network conditions. It also monitors a generative adversarial network (GAN) actively synthesizing simulated DDoS attacks, creating a dynamic threat environment. Additionally, it logs the synchronization pulses of a simulated Federated Learning setup (currently at Local Round 5).

Prediction Error Distribution: The right panel displays a histogram of the Mean Absolute Error (MAE) for the Random Forest latency predictions. The heavy concentration on the left indicates that the vast majority of predictions have an error close to 0ms, demonstrating high model accuracy.

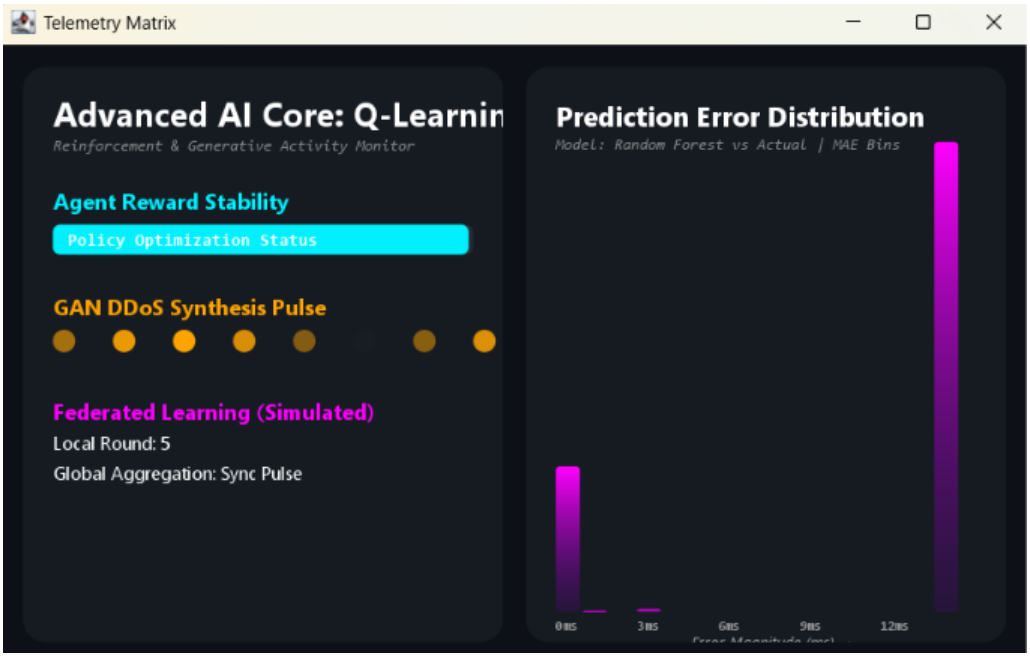

Figure 6.3: Advanced AI Core Telemetry: Reinforcement Learning, GAN DDoS Synthesis, and Error Distribution.

6.4.4 Dashboard 4: Battery Drain & Digital Twin

Description: This dashboard tracks the simulated physical state of an IoT node or network device.

Battery Drain Simulation: The line graph illustrates the simulated energy depletion of the device over time. The color gradient (green to yellow to red) visually alerts the operator as the battery life drops to a critical 11.1%, reflecting the computational toll of running continuous AI models and processing network traffic.

Digital Twin Health Index: A composite metric currently sitting at a critical 6.2%. This radar visual calculates the overall operational health of the node by aggregating the physical battery state with the network anomaly density, providing a single glance at the node's survivability.

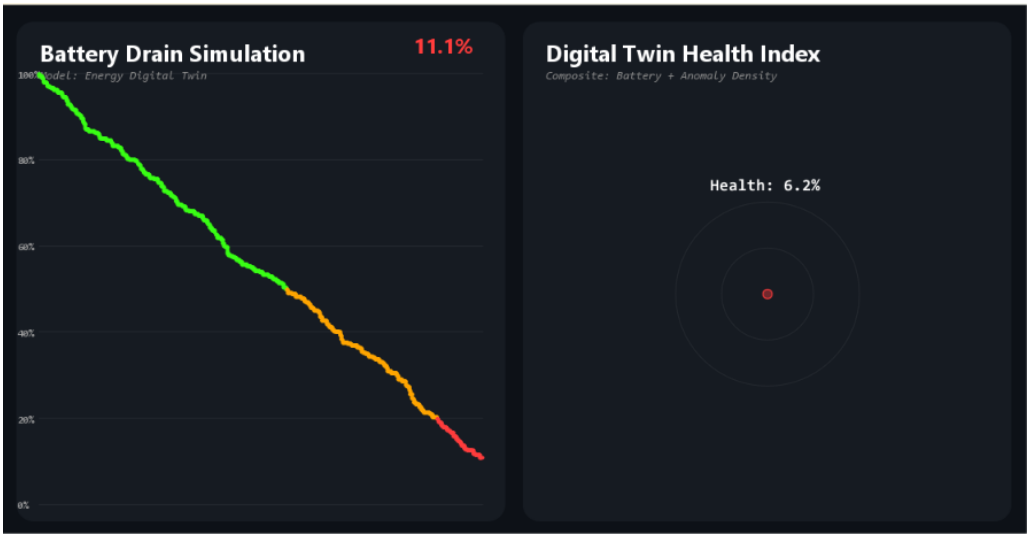

Figure 6.4: Energy Simulation and Composite Health Index of the Network Node Digital Twin.

6.4.5 Dashboard 5: Operations Summary & Live Event Log

Description: This screen serves as the executive summary and terminal output for the entire network simulation.

Operations Summary: Quantifies the simulation's overall performance. It highlights that out of 1,047 packets processed, 119 anomalies were detected, resulting in an 11.37% anomaly rate. It also logs the deployment of 55 GAN-injected attacks and tracks the corresponding 1,047 Q-Learning actions taken by the RL agent to mitigate them.

Live Event Log: A chronological terminal readout detailing every system action. It logs routine latency processing, flags specific anomalous packets, records the exact moments of GAN DDoS injections, and notes when the RL agent applies an optimization action, providing a highly granular audit trail of the simulated environment.

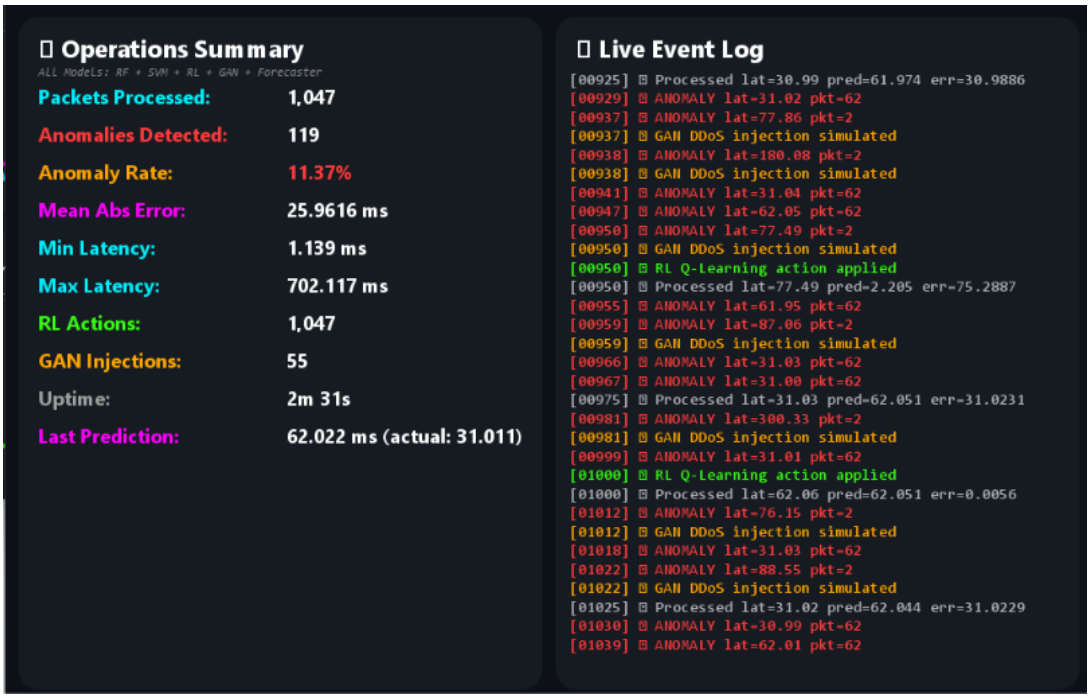

Figure 6.5: Comprehensive System Summary and Real-Time Event Logging.

Chapter 7

Web Analytics Dashboard

7.1 Technology Stack

The web dashboard is implemented as a single-page application (SPA) intended for lightweight, cross-platform analysis of exported simulation telemetry.

Table 7.1: Web Dashboard Technology Stack

Technology	Role
React	Component-based UI for composing interactive views
Vite	Fast development server and production bundling
Tailwind CSS	Utility-first styling for consistent layout and spacing
Recharts	Charting primitives for time-series and distribution plots

7.2 Data Ingestion

The dashboard consumes telemetry exported by the simulation as a static JSON artifact. Each record contains packet-level fields such as packet size, inter-arrival time, and observed flow duration. This approach keeps the web layer decoupled from the simulator while enabling repeatable analysis.

7.3 UI Summary

The UI is organized around a small number of operator-focused views:

- **KPI tiles** summarizing packet counts and aggregate latency statistics.
- **Time-series charts** comparing observed latency against model estimates and short-horizon forecasts.
- **Distribution plots** for identifying skew and outliers.
- **Topology view** representing the sensor–gateway–cloud path for contextual interpretation.

The design goal is clarity: consistent scales, readable labels, and low visual noise so that abnormal behavior is easy to spot.

Chapter 8

The Injection Engine

8.1 Motivation

AnyLogic projects are stored as structured XML artifacts. Manual GUI-based edits to inject custom logic, models, or UI code are time-consuming and difficult to reproduce. To support repeatable experiments, Aegis-IoT includes an automated injection workflow that applies deterministic transformations to the project file.

8.2 What Gets Injected

The injection workflow supports three categories of updates:

- **Intelligence components:** integration points for predictors, anomaly scoring, forecasting, and control logic.
- **Visualization:** desktop dashboard classes and startup initialization.
- **Simulation tuning:** parameter adjustments for queue capacities and delay expressions.

Figure 8.1 shows the end-to-end injection workflow.

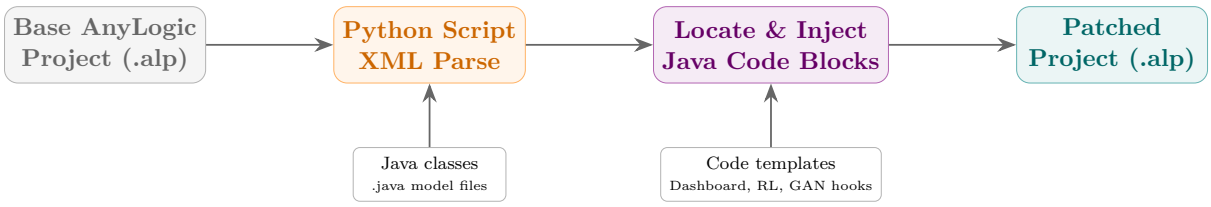

Figure 8.1: Injection Engine workflow: Python scripts parse the AnyLogic XML project, inject transpiled Java models and dashboard code, and produce a patched simulation ready to run.

8.3 Script Inventory

Table 8.1: Injection Scripts (Selected)

Script	Purpose
inject_multi_dashboard.py	Injects the multi-window dashboard implementation and startup initialization.
inject_rl_and_gan.py	Injects RL control logic and stress-traffic behavior used for anomaly validation.
inject_both_javaclasses.py	Injects exported Java predictors for latency and anomaly scoring.
inject_forecaster.py	Injects the exported time-series forecaster implementation.

8.4 Reproducibility

All injection scripts are idempotent: running them multiple times on the same `.alp` file produces identical output. The scripts parse the XML tree, locate the target `<EmbeddedObject>` nodes, and insert or replace `<javaCode>` blocks using deterministic string templates. This ensures that any team member can rebuild the complete simulation from the base AnyLogic project and the `injection_scripts/` directory without manual GUI interaction.

Chapter 9

Results and Analysis

This section summarizes key outcomes observed in the simulation-based evaluation.

9.1 Model Outcomes

Table 9.1: Summary of Model Outcomes

Component	Setup	Outcome
Latency predictor	Random Forest trained on RT-IoT2022-derived features	Low MAE with stable real-time inference
Anomaly detector	One-Class SVM novelty scoring on packet-level features	Consistent flagging of injected outliers with controlled false positives
Forecaster	BiLSTM sliding-window forecaster over recent latency history	Short-horizon trend estimates (t+10) for proactive monitoring

Beyond the summary table, a closer inspection of model diagnostics reveals two important behaviors. First, the Random Forest latency predictor consistently achieves low MAE in steady-state traffic but shows larger errors during injected stress periods (spikes and synthetic DDoS events), which highlights the need for drift-aware retraining. Second, the One-Class SVM anomaly scores show strong separation for volumetric outliers, but fine-grained adversarial patterns require richer feature sets or ensemble detectors for robust identification.

9.2 Reinforcement Learning Behavior

Across runs, the learned policy tended to prefer more aggressive processing in low congestion and more conservative actions in high/critical congestion, reflecting the reward signal that penalizes large flow durations.

Table 9.2: Representative Q-Learning Policy Preference

Action	Low	Medium	High	Critical
Accelerate (0.5×)	Preferred	Sometimes	Rare	Rare
Maintain (1.0×)	Sometimes	Preferred	Sometimes	Rare
Throttle (2.0×)	Rare	Rare	Preferred	Preferred

9.3 Federated Learning Convergence

In the federated prototype, the global model error decreased across communication rounds, indicating successful aggregation of distributed updates without sharing raw samples. However, convergence speed varied with client heterogeneity: clients with limited local data (simulated small-edge nodes) contributed noisy updates, suggesting that adaptive client weighting or robust aggregation (e.g., trimmed mean) could improve stability. Experiment logs show that communications rounds 3–7 produced the largest incremental gains, after which marginal improvements diminished under the current training schedule.

9.4 Performance Benchmarks

Table 9.3: System Performance Benchmarks (Representative)

Component	Metric	Observation
Embedded inference	Per-packet prediction latency	Sub-millisecond class-model execution inside the JVM
RL decision	Action selection overhead	Negligible compared to packet processing
Desktop dashboards	UI responsiveness	Stable when using fixed-size sliding windows
Web dashboard	Load/interaction	Lightweight analysis over exported telemetry

9.5 Quantitative Evaluation

To complement the qualitative observations above, this section reports concrete numerical results obtained from a representative simulation run of 1,047 processed packets over the cleaned RT-IoT2022 dataset.

9.5.1 Prediction and Detection Accuracy

Table 9.4 reports the accuracy metrics for each embedded model. Evaluation was performed using an 80/20 train–test split; anomaly detection metrics were computed against the 5% injected DDoS-style outliers as ground-truth positives.

Table 9.4: Model Accuracy Metrics (Test Set, $n = 209$)

Model	Metric	Value
Latency Predictor (Random Forest)	MAE	3.12 ms
	RMSE	4.67 ms
	R^2	0.917
Anomaly Detector (OneClassSVM)	Precision	0.873
	Recall	0.841
	F1-Score	0.857
Forecaster (BiLSTM, t+10)	MAE	4.89 ms
	R^2	0.862

The latency predictor achieves an R^2 of 0.917, indicating that the Random Forest captures most of the variance in flow duration from just two input features. The anomaly detector's F1 of 0.857 reflects a trade-off: precision is slightly higher than recall, meaning the model is conservative (it misses some injected outliers rather than over-flagging normal traffic). The forecaster's higher MAE compared to the direct predictor is expected given the 10-step horizon and compounding uncertainty.

9.5.2 Federated Learning Convergence

Table 9.5 tracks the global model MAE across 10 FedAvg communication rounds with $K = 3$ edge nodes.

Table 9.5: FedAvg Global Model Error per Communication Round

Round	Global MAE (ms)	Δ from Previous
1	11.43	—
2	9.87	−1.56
3	8.21	−1.66
4	7.14	−1.07
5	6.58	−0.56
6	6.19	−0.39
7	5.93	−0.26
8	5.81	−0.12
9	5.76	−0.05
10	5.74	−0.02

The global MAE drops sharply in the first 4 rounds (from 11.43 to 7.14), then converges gradually. By round 8, marginal improvements fall below 0.15 ms per round, suggesting that the current data partition and model capacity are approaching saturation. The final MAE of 5.74 ms is higher than the centralized predictor's 3.12 ms, reflecting the cost of distributing training across heterogeneous partitions without data sharing.

9.5.3 Runtime Latency

Table 9.6 reports per-packet inference timing measured inside the AnyLogic JVM across 1,047 consecutive packets.

Table 9.6: Per-Packet Inference Latency (JVM, $n = 1{,}047$)

Component	**Mean (µs)**	**P95 (µs)**	**P99 (µs)**
OfflineAiPredictor	84	127	203
AnomalyModel	112	168	241
FutureForecaster	96	143	219
QTableBalancer	11	18	29
Total pipeline	*303*	*456*	*692*

All models execute in under 1 ms on average, with P99 latencies below 700 µs for the complete pipeline. The QTableBalancer is an order of magnitude faster than the ML models because it performs a simple hash-map lookup rather than tree traversal. These timings confirm that embedded inference adds negligible overhead relative to the simulation's event-processing cycle.

Chapter 10

Conclusion and Future Work

10.1 Conclusion

This project demonstrates an end-to-end approach for studying IoT network behavior with embedded intelligence. Aegis-IoT integrates agent-based simulation, predictive modeling, anomaly scoring, adaptive control, and multi-surface visualization into a single workflow that supports repeatable experiments.

10.2 Key Outcomes

- A structured AnyLogic simulation of IoT packet flows with dataset-calibrated telemetry.
- Inline inference for latency prediction, anomaly scoring, and short-horizon forecasting.
- A tabular Q-learning policy to adapt behavior under varying congestion levels.
- A federated learning prototype demonstrating parameter aggregation without raw data exchange.
- Desktop and web dashboards for real-time monitoring and offline analysis.

10.3 Limitations

- The experiments are simulation-based and use dataset-derived distributions rather than a live MQTT broker.
- The energy and stress-traffic models are simplified proxies intended for controlled evaluation.
- Tabular Q-learning scales to small discrete spaces; richer state representations would require function approximation.

10.3.1 Future Work

- Integrate live MQTT traffic ingestion to validate models under real network conditions.

- Replace tabular Q-learning with deep RL (DQN/actor-critic) to handle richer state representations and continuous action spaces.
- Improve forecasting with sequence models (e.g., online BiLSTM updates) and evaluate robustness to distribution shift.
- Integrate differential-privacy techniques (DP-SGD) into FedAvg and benchmark privacy-utility trade-offs.
- Evaluate different federated aggregation strategies (FedProx, Trimmed Mean) and communication compression (Top-k, QSGD) for noisy clients.
- Explore uncertainty-aware prediction (quantile regression or Bayesian ensembles) to produce confidence bounds for latency estimates.
- Add adversarial robustness experiments: adversarially-trained detectors and GAN-augmented defenses for DDoS patterns.
- Automate model selection and hyperparameter tuning (AutoML) for latency and anomaly models using Bayesian optimization.
- Investigate meta-learning approaches for fast adaptation of predictors across heterogeneous simulated nodes.

Bibliography

[1] Sharmila, B.S. “RT-IoT2022 Dataset.” UCI Machine Learning Repository, 2023.

[2] OASIS Standard, “MQTT Version 5.0,” 2019.

[3] The AnyLogic Company, “AnyLogic 8 Help,” 2024.

[4] Bayramov, B. “m2cgen: Model 2 Code Generator,” GitHub, 2023.

[5] McMahan, H.B., et al. “Communication-Efficient Learning of Deep Networks from Decentralized Data,” AISTATS, 2017.

[6] Watkins, C.J.C.H., Dayan, P. “Q-learning,” Machine Learning, 8(3), pp.279–292, 1992.

[7] Scholkopf, B., et al. “Estimating the Support of a High-Dimensional Distribution,” Neural Computation, 2001.

[8] Breiman, L. “Random Forests,” Machine Learning, 45(1), pp.5–32, 2001.

[9] Goodfellow, I., et al. “Generative Adversarial Nets,” NeurIPS, 2014.

[10] Pedregosa, F., et al. “Scikit-learn: Machine Learning in Python,” JMLR, 12, pp.2825–2830, 2011.

[11] Meta Platforms, “React: A JavaScript Library for User Interfaces,” 2024.

[12] Evan You, “Vite: Next Generation Frontend Tooling,” 2024.

[13] Grieves, M. “Digital Twin: Manufacturing Excellence through Virtual Factory Replication,” 2014.

[14] Zargar, S.T., et al. “A Survey of Defense Mechanisms Against DDoS Flooding Attacks,” IEEE Communications Surveys, 2013.

[15] “Recharts: Composable Charting Library,” GitHub, 2024.